Project Title

Efficient String Pattern Matching Using the Z-Algorithm: A Comparative Performance Analysis

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
A. Charan Teja (2520030418)
Sk. Juneeth (2520030435)
D. Venkata Mahith (2520030497)

Team Number: 21

Section: 4

Under the guidance of

Mrs. J. Yamini Devi



Assistant Professor, Department of CSE

Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



DECLARATION

The Mini Project Report entitled "**Efficient String Pattern Matching Using the Z-Algorithm: A Comparative Performance Analysis"** is a record of Bonafide work of **A. Charan Teja – 2520030418, Sk. Juneeth – 2520030435, D. Venkata Mahith – 2520030497,** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

A. Charan Teja – 2520030418

Sk. Juneeth – 2520030435

D. Venkata Mahith – 2520030497

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled "**Efficient String Pattern Matching Using the Z-Algorithm: A Comparative Performance Analysis"** is a bonafide work done and submitted by **A. Charan Teja – 2520030418, Sk. Juneeth – 2520030435, D. Venkata Mahith – 2520030497** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Mrs. J. Yamini Devi, Assistant Professor, Department of Computer Science and Engineering, which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P. Venkateshwar Rao**, Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

A. Charan Teja – 2520030418

Sk. Juneeth – 2520030435

D. Venkata Mahith – 2520030497

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	14
4.	Problem Statement and Objectives	16
5.	Hardware and Software requirements	18
6.	Implementation	19
7.	Results & Discussion	25
8.	Conclusion & Future Scope	32
9.	References	34

1.ABSTRACT

String pattern matching, which means locating all occurrences of a pattern P within a text T, is a core problem in computer science. It underlies text editors, search engines, bioinformatics and intrusion-detection systems.

This project implements the Z-Algorithm for exact pattern matching. It builds a Z-array in linear O(n + m) time using a simple two-pointer window (l, r), which avoids the character-by-character re-scanning that makes the naive method slow.

The Z-Algorithm is compared against three classical approaches, namely Naive Search, Rabin–Karp and Knuth–Morris–Pratt (KMP), through empirical benchmarking across varying text sizes and pattern lengths. The project shows how a simple preprocessing idea gives a linear-time guarantee and serves as an easy-to-teach introduction to efficient string algorithms.

Team Name: Team 21

Team Logo (if any): Not applicable

Team Members:

1. A. Charan Teja, 2520030418
2. Sk. Juneeth, 2520030435
3. D. Venkata Mahith, 2520030497

2. INTRODUCTION

Pattern matching is the task of finding every occurrence of a pattern P of length m inside a text T of length n. It is a building block for search engines, spell-checkers, DNA and protein sequence analysis, and network intrusion detection, where the text can run to millions or billions of characters and searches are repeated many times.

The naive method places the pattern at every text position and compares character by character. In the worst case this takes O(n·m) time, because after a mismatch it throws away everything it learned and starts again from the next position. This becomes impractical for large inputs such as genomic sequences or web-scale documents.

The Z-Algorithm (formalised by Gusfield, 1997) removes this repeated work. For a string S it computes a Z-array in which Z[i] is the length of the longest substring starting at position i that is also a prefix of S. To search for P in T, the algorithm forms the combined string S = P + "$" + T, where "$" is a separator that does not occur in either string. A position i inside the text part of S is the start of a match exactly when Z[i] equals m, and the match is reported at index i − m − 1 of T.

The Z-array is built in a single left-to-right pass by keeping a window [l, r], the rightmost segment found so far that matches a prefix of S. When the current position i lies inside this window, the value of Z[i] is copied from an earlier position instead of being recomputed, and explicit comparison starts only beyond the window. Since r never moves backwards, the total work is O(n + m).

The motivation for this project is to study whether the Z-Algorithm can give the efficiency of KMP and Rabin–Karp while remaining simple to implement. We implement the Z-Algorithm from first principles, implement the three baselines, and compare them in terms of complexity and measured running time. The rest of this report covers the literature survey, the problem statement and objectives, the hardware and software requirements, the implementation, the results and discussion, and the conclusion with future scope.

Theoretical Background

2.1 Basic terminology

Let Σ be a finite alphabet and let S[0..n−1] be a string of length n over Σ. A substring S[i..j] is a contiguous block of characters. A prefix of S is a substring that starts at position 0, and a suffix is a substring that ends at position n−1. A border of S is a non-empty proper prefix of S that is also a suffix of S; for example, "aab" is a border of "aabxaab". Two strings are equal if they have the same length and agree at every position.

The exact pattern-matching problem is defined as follows. Given a text T of length n and a pattern P of length m with m ≤ n, find all indices i with 0 ≤ i ≤ n − m such that T[i..i+m−1] = P. An algorithm is judged by the number of character comparisons it makes and by the extra

memory it needs. The naive algorithm needs up to $(n - m + 1) \cdot m$ comparisons, which is $O(n \cdot m)$. The lower bound for any algorithm that must read the whole text is $\Omega(n)$, so a running time of $O(n + m)$ is optimal up to a constant factor.

2.2 The Z-array

For a string S of length N, the Z-array is defined for every position i with $1 \leq i < N$ as

Z[i] = the largest k such that S[0..k−1] = S[i..i+k−1].

In words, Z[i] is the length of the longest substring that starts at i and is also a prefix of S. The value Z[0] is not meaningful (it would always be N) and is conventionally set to 0. If Z[i] = k, then the characters S[i..i+k−1] equal the first k characters of S, and the character S[i+k] (if it exists) differs from S[k]. The table below shows the Z-array of S = "aabxaab".

Table 2.1: Z-array of S = "aabxaab"

Index i	0	1	2	3	4	5	6
S[i]	a	a	b	x	a	a	b
Z[i]	–	1	0	0	3	1	0

For i = 4 the substring "aab" equals the prefix "aab" and the string then ends, so Z[4] = 3. For i = 5 only "a" matches, because S[6] = b differs from S[1] = a, so Z[5] = 1.

A direct computation of every Z[i] by comparing from scratch takes $O(N^2)$ time in the worst case, for example for $S = a^N$. The Z-Algorithm computes the whole array in O(N) time by reusing the values that are already known.

2.3 The Z-box and the two cases

While scanning from left to right, the algorithm keeps a window [l, r), called the Z-box. It is the match segment found so far that reaches furthest to the right: S[l..r−1] equals the prefix S[0..r−l−1], and r is as large as possible among all positions processed so far. The Z-box is the information that is reused. When the algorithm moves to position i there are two cases.

Case 1: i ≥ r. Position i lies outside the window, so nothing is known about it. The algorithm compares S[i], S[i+1], … with S[0], S[1], … one by one until a mismatch or the end of the string, and sets Z[i] to the number of matches. If Z[i] > 0, the window becomes l = i and r = i + Z[i].

Case 2: i < r. Position i lies inside the window. Because S[l..r−1] equals S[0..r−l−1], the substring starting at i matches the substring starting at k = i − l, as far as the window extends. Hence Z[i] is at least min(Z[k], r − i). There are two sub-cases. If Z[k] < r − i, the match starting at k ends inside the window, so the match at i ends at the same relative place and Z[i] = Z[k] exactly, with no comparison at all. If Z[k] ≥ r − i, the match at i reaches the right end of the window and may continue; the algorithm starts comparing from S[r] against S[r − i] and extends the window.

Table 2.2: Step-by-step construction of the Z-array for S = "aabxaab"

i	l	r (before)	Case	Action	Z[i]
1	0	0	1 (i ≥ r)	Compare from scratch: "a" matches, then b ≠ a. New window [1, 2)	1
2	1	2	1 (i ≥ r)	S[2] = b ≠ S[0] = a	0
3	1	2	1 (i ≥ r)	S[3] = x ≠ S[0] = a	0
4	1	2	1 (i ≥ r)	"aab" matches up to the end. New window [4, 7)	3
5	4	7	2 (i < r)	k = 1, Z[1] = 1 < r − i = 2, so copy Z[1]; no comparison	1
6	4	7	2 (i < r)	k = 2, Z[2] = 0 < r − i = 1, so copy Z[2]; no comparison	0

2.4 Algorithm (pseudo-code)

```
Z-ARRAY(S):                          // N = length of S
    Z[0..N-1] = 0;  l = 0;  r = 0
    for i = 1 to N-1:
        if i < r:
            Z[i] = min(r - i, Z[i - l])      // reuse the window
        while i + Z[i] < N and S[Z[i]] == S[i + Z[i]]:
            Z[i] = Z[i] + 1                  // explicit comparison
        if i + Z[i] > r:
            l = i;  r = i + Z[i]             // extend the window
    return Z

Z-SEARCH(T, P):                      // m = |P|
    S = P + "$" + T
    Z = Z-ARRAY(S)
    for i = m+1 to |S|-1:
        if Z[i] >= m:  report match at index i - m - 1 of T
```

2.5 Pattern matching with the Z-array

To search for P in T, the two strings are joined as S = P + "$" + T, where "$" is a separator symbol that occurs in neither P nor T. For a position i > m in S, the value Z[i] tells how long the text starting at that position agrees with the beginning of the pattern. A full match occurs exactly when Z[i] = m, and the corresponding start index in the text is i − (m + 1).

The separator is essential. The character "$" occurs only once in S, at position m. A match of length greater than m starting at some i > m would require S[i+m] to equal S[m] = "$", which is impossible because "$" does not appear after position m. Therefore Z[i] ≤ m for every i > m, and the test Z[i] = m is exact; without the separator, a match could run on into the text and give values larger than m. In the implementation the test is written Z[i] ≥ m. Whenever Z[i] ≥ m, the m characters starting at position i equal the first m characters of S, which is the pattern, so the test is still correct if the user happens to supply a text or pattern that contains "$".

2.6 Correctness

The correctness follows from an invariant: at the start of the iteration for position i, all values Z[1..i−1] are correct and [l, r) is the Z-box with the largest right end r among the positions 1..i−1.

In Case 1 the value is found by direct comparison and is therefore correct. In Case 2 the equality S[l..r−1] = S[0..r−l−1] implies that S[i..r−1] = S[k..r−l−1] with k = i − l. If Z[k] < r − i, the mismatch that ends the match at k lies inside the window, and it is mirrored at position i, so Z[i] = Z[k]. Otherwise the first r − i characters certainly match and the explicit comparison continues from S[r], which finds the exact value. Updating the window to the one with the larger right end keeps the invariant for the next position.

2.7 Time and space complexity

The running time is analysed by counting character comparisons. Every comparison that succeeds, or that happens in the while-loop, starts at or beyond the position r and, when it succeeds, moves the right end r of the window one step to the right. Since r never decreases and cannot exceed N, there are at most N successful comparisons in total. Each position i causes at most one failed comparison. Hence the total number of comparisons is at most 2N, and the other work per position (the min and the copy) is constant. The Z-array is therefore built in O(N) time, and with N = n + m + 1 the search takes O(n + m) time in the best, average and worst case.

The space needed is one integer array of size N = n + m + 1, that is, O(n + m). If memory is critical, the Z-array of the pattern alone can be computed first, and the text can then be scanned while keeping only the current window, which reduces the extra space to O(m).

2.8 Theory of the baseline algorithms

Naive search. The pattern is aligned at each of the n − m + 1 text positions and compared from left to right. After a mismatch the pattern is moved by one position and everything is compared again. The worst case occurs for texts such as $T = a^n$ and $P = a^{m-1}b$, where almost m characters match at every alignment, giving about (n − m + 1)·m comparisons, i.e., O(n·m). On random text over a large alphabet a mismatch usually occurs at the first or second character, so the expected time is close to O(n).

Rabin–Karp. A string s of length m is mapped to a number by the polynomial hash H(s) = (s[0]·b^(m−1) + s[1]·b^(m−2) + … + s[m−1]) mod q, where b is a base (256 in our implementation) and q is a large prime ($10^9 + 7$). The hash of the next window is obtained from the previous one in constant time by the rolling update H' = ((H − s[0]·b^(m−1))·b + s[m]) mod q. A window whose hash equals the hash of the pattern is checked character by character, since different strings can have equal hashes (a spurious hit or collision). With a good hash the expected time is O(n + m); if many collisions occur the time degrades to O(n·m).

Knuth–Morris–Pratt. KMP precomputes the prefix function (LPS array) of the pattern: π[i] is the length of the longest proper prefix of P[0..i] that is also a suffix of P[0..i]. After a mismatch at pattern position j, the search continues from position π[j−1] of the pattern instead of restarting, so the text pointer never moves backwards. Each text character is processed in amortised constant time, giving O(n + m) time and O(m) extra space.

2.9 Z-array versus prefix function

The Z-array and the prefix function of KMP carry the same information about the borders and self-overlaps of a string, and each can be converted into the other in linear time. They differ in how the information is stored and used, as summarised below.

Table 2.3: Comparison of the Z-array and the KMP prefix function

Aspect	Z-array (Z-Algorithm)	Prefix function (KMP)
Meaning of the value	Length of the longest match between the suffix starting at i and the whole string S	Length of the longest proper prefix of P[0..i] that is also its suffix
Computed over	Combined string P + "$" + T	Pattern P only
Preprocessing idea	Two-pointer window [l, r) reused from the left	Fallback along the chain of borders after a mismatch
Match test	Z[i] = |P|	Matched length reaches |P|
Time	O(n + m)	O(n + m)
Extra space	O(n + m) (O(m) with the variant in 2.7)	O(m)
Ease of understanding	Direct: each value is a plain match length	Indirect: values describe borders and fallbacks

2.10 Applications and limitations

Besides searching for a pattern, the Z-array solves several related problems in linear time. A string S of length N has a period p (that is, S[i] = S[i+p] for all valid i) exactly when Z[p] = N − p, so the smallest period and the border structure of a string can be read directly from the array. It is also used for finding repeated prefixes, for counting occurrences, for string compression checks and in bioinformatics for locating a motif in a sequence.

The Z-Algorithm has limitations. It finds exact matches only and does not tolerate errors, it searches for one pattern at a time (multi-pattern search needs a structure such as the Aho–Corasick automaton), and in its basic form it needs memory proportional to the length of the text plus the pattern. Its running time also does not exploit favourable inputs: unlike Boyer–Moore, it always reads every character of the text, so it cannot be sub-linear in practice.

2.11 Why the period test works

The relation between the Z-array and periods, mentioned above, follows directly from the definition. A string S of length N has period p (with $0 < p < N$) if S[i] = S[i + p] for every i from 0 to N − p − 1. This says exactly that the prefix S[0..N−p−1] equals the suffix S[p..N−1]. Since Z[p] is the longest match between the suffix starting at p and the prefix of S, and that suffix has length N − p, the condition is the same as Z[p] = N − p. For S = "aabxaab" we get Z[4] = 3 = 7 − 4, so p = 4 is a period, and the border "aab" has length N − p = 3. The smallest such p is the smallest period of the string.

2.12 Worked example of Rabin–Karp hashing

To make the hashing idea concrete, consider a text of decimal digits T = 3141592653 and a pattern P = 159, with base b = 10 and modulus q = 13. The hash of the pattern is 159 mod 13 = 3. Each window of three digits is hashed and compared; the rolling update gives the next hash from the previous one in constant time. The table lists every window.

Table 2.4: Rabin–Karp on T = 3141592653 with P = 159 (b = 10, q = 13)

Start index	Window	Hash (window mod 13)	Equal to pattern hash (3)?	Real match?
0	314	2	No	No
1	141	11	No	No
2	415	12	No	No
3	159	3	Yes	Yes
4	592	7	No	No
5	926	3	Yes	No (spurious hit)
6	265	5	No	No
7	653	3	Yes	No (spurious hit)

The windows at indices 5 and 7 have the same hash as the pattern although they are different strings. These spurious hits must be rejected by a direct comparison, which is why a weak hash function or a small modulus increases the running time. Choosing a large prime modulus, as in our implementation ($q = 10^9 + 7$), makes such collisions rare.

2.13 Worked example of the KMP prefix function

For the pattern P = "aabxaab" the prefix function π[i] is the length of the longest proper prefix of P[0..i] that is also a suffix of P[0..i]. The values are computed left to right, and after a mismatch the candidate length falls back through earlier values of π.

Table 2.5: Prefix function (LPS array) of P = "aabxaab"

Index i	0	1	2	3	4	5	6
P[i]	a	a	b	x	a	a	b
π[i]	0	1	0	0	1	2	3

Comparing Table 2.5 with Table 2.1 shows the connection between the two arrays: at the last position π[6] = 3 is the length of the border "aab", and the Z-array finds the same border as Z[4] = 3. Searching T = "aabxaabxcaabxaab" with this table finds the pattern at indices 0 and 9. After the first match the search does not restart: it continues with the length π[6] = 3, because the border "aab" is already matched. At text index 8 the character c differs from P[4], so the matched length falls back to π[3] = 0 and the scan simply moves on, finding the second match at index 9.

2.14 Boyer–Moore and sub-linear searching

Boyer–Moore compares the pattern with the text from right to left. When a mismatch occurs, two rules decide how far the pattern may be shifted. The bad-character rule aligns the mismatching text character with its rightmost occurrence in the pattern, or moves the pattern completely past it if the character does not occur in the pattern. The good-suffix rule uses the part of the pattern that has already matched to find the next alignment that is consistent with it. For large alphabets and long patterns the shifts are often close to m, so only about n/m characters are examined and the algorithm is sub-linear in practice. In its original form the worst case is O(n·m) when the pattern occurs many times, but adding Galil's rule makes it linear. This explains why Boyer–Moore is widely used in text editors, while the Z-Algorithm and KMP are preferred when a worst-case guarantee and a simple implementation are the priorities.

2.15 Multiple patterns and approximate matching

The Z-Algorithm solves the single-pattern exact problem. When k patterns must be found, running it k times costs O(k·n); the Aho–Corasick automaton instead handles all patterns in one pass, and the two ideas can be combined when some patterns are searched with an automaton and others verified with Z-values. When errors must be tolerated, the problem is defined through the edit distance, the minimum number of insertions, deletions and substitutions that turn one string into another, which is computed by dynamic programming in O(n·m) time. Z-values can still be used as a filter in such methods, because a pattern with at most k errors must contain a piece of length ⌊m/(k+1)⌋ that matches exactly.

2.16 Summary of asymptotic notation used

The report uses the following notation. O(f(n)) is an upper bound on the growth of the running time, Ω(f(n)) is a lower bound and Θ(f(n)) means that both bounds hold. The symbols n and m denote the lengths of the text and the pattern, σ the alphabet size, and N = n + m + 1 the length of the combined string. Table 2.6 summarises the cost measures of the four algorithms studied.

Table 2.6: Summary of cost measures

Algorithm	Preprocessing time	Search time (average)	Search time (worst)	Extra space
Naive	O(1)	O(n) (random text)	O(n·m)	O(1)
Rabin–Karp	O(m)	O(n + m)	O(n·m)	O(1)
KMP	O(m)	O(n)	O(n)	O(m)
Z-Algorithm	**O(m) for the pattern part**	**O(n + m)**	**O(n + m)**	**O(n + m)**

3. LITERATURE SURVEY

1. Knuth, Morris and Pratt (KMP), 1977. KMP introduced a failure-function (prefix-function) table, computed from the pattern, that tells the search how far the pattern can shift after a mismatch without re-examining characters already matched. It guarantees O(n + m) time in the worst case and is the standard linear-time baseline against which other exact-matching algorithms are compared.

2. Boyer and Moore, 1977. Boyer and Moore scan the pattern from right to left and use the bad-character and good-suffix heuristics to skip large parts of the text. The algorithm is often sub-linear in practice, which makes it popular in text editors, although its heuristics are more complex to implement correctly.

3. Rabin and Karp, 1987. Rabin and Karp compare rolling hash values of the pattern and of each text window, and verify a character-by-character match only when the hashes agree. This gives O(n + m) average time and extends naturally to searching for many patterns at once, but hash collisions can push the worst case to O(n·m).

4. Gusfield, Algorithms on Strings, Trees and Sequences, 1997. Gusfield presented the Z-Algorithm and the Z-array as a general-purpose string-processing primitive. Many problems, such as exact matching and finding borders and repeats, can be reduced to computing Z-values, and the method needs only a single linear scan with a two-pointer window.

5. Aho and Corasick, 1975. Aho and Corasick built a finite automaton (a trie with failure links) from a whole set of patterns, so that all of them can be searched in a single pass over the text. The running time is linear in the length of the text plus the total length of the patterns plus the number of matches. It is the standard answer to the multi-pattern problem that the Z-Algorithm alone does not solve.

6. Suffix trees: Weiner (1973), McCreight (1976) and Ukkonen (1995). A suffix tree stores all suffixes of a text in a compressed trie and can be built in linear time. After this preprocessing of the text, any pattern of length m can be searched in O(m) time, which suits repeated queries on a fixed text. The price is a large memory footprint and a complex construction, which is why the simpler online algorithms are preferred for one-time searches.

7. Crochemore and Perrin, 1991 (Two-Way algorithm). The Two-Way algorithm finds all occurrences in O(n) time using only constant extra space, by splitting the pattern at a critical position and scanning in both directions. It shows that linear time does not require linear memory, a direction that is relevant to the memory use of the Z-Algorithm discussed in this report.

8. Navarro, 2001 (approximate string matching). Navarro's survey of approximate matching reviews algorithms that allow a bounded number of insertions, deletions and substitutions (edit distance). These methods are needed for noisy data such as typing errors and sequencing errors,

and they mark the boundary beyond which exact algorithms such as the Z-Algorithm are not sufficient.

9. Comparative benchmarking studies, 2015–2023. Later studies applied these algorithms to bioinformatics and big-data text search and compared them empirically. They highlight the Z-Algorithm's simplicity for exact matching while showing that practical speed depends on the alphabet, the pattern length and the text structure.

Table 3.1: Comparison of exact and related string-matching algorithms from the literature

Algorithm	Year	Preprocessing	Search time (worst case)	Extra space	Remark
Naive	–	None	O(n·m)	O(1)	Simplest; no preprocessing
Knuth–Morris–Pratt	1977	O(m)	O(n)	O(m)	Text pointer never moves back
Boyer–Moore	1977	O(m + σ)	O(n·m) in the original form; best case O(n/m)	O(m + σ)	Often sub-linear in practice
Rabin–Karp	1987	O(m)	O(n·m); O(n + m) expected	O(1)	Hash based; multi-pattern capable
Z-Algorithm	1997	Z-array of P$T	O(n + m)	O(n + m)	Simple, deterministic
Aho–Corasick	1975	O(total pattern length)	O(n + matches)	O(total pattern length)	Many patterns at once
Two-Way	1991	O(m)	O(n)	O(1)	Constant extra space

Summary: *across the literature, exact-matching algorithms trade preprocessing complexity for linear-time guarantees. The Z-Algorithm achieves this with the simplest preprocessing step among them, which motivates its use in this project.*

4. Problem Statement and Objectives

Problem Statement:

Given a text T of length n and a pattern P of length m over some alphabet, find all positions at which P occurs in T, in time that is linear in n + m and without repeatedly checking the same characters.

Problem Description:

The problem is to search for a pattern quickly in a large text without re-checking characters that have already been compared. The naive method needs O(n·m) time in the worst case. Algorithms such as KMP, Rabin–Karp and the Z-Algorithm provide more efficient solutions. Our project focuses on the Z-Algorithm and studies whether it can provide this efficiency while remaining simple to implement.

Objectives:

Purpose of the project: The project was started to show how a Z-array lets us match patterns in linear time, and to measure how this compares with other well-known approaches. The following objectives were framed to achieve this.

1. Z-Algorithm Implementation: Implement the Z-Algorithm for exact pattern matching from first principles.

2. Baseline Algorithms: Implement Naive, Rabin–Karp and KMP as performance baselines.

3. Empirical Comparison: Compare execution time and preprocessing overhead across varying text sizes and pattern lengths.

4. Practicality Assessment: Assess the practicality of the Z-Algorithm for real-world tasks such as text search and substring matching.

5. Visualisation: Visualise and document performance trends using comparative tables and charts.

Input and Output Specification:

Input: a text T of length n and a pattern P of length m, both given as character strings with $1 \leq m \leq n$. The comparison is exact and case-sensitive.

Output: the list of all start indices i (counted from 0) at which P occurs in T, in increasing order, including overlapping occurrences. The list is empty if P does not occur in T.

Assumptions and Constraints:

- **Exact matching only:** a position is reported only if all m characters agree; approximate matching is outside the scope.

- **Single pattern:** one pattern is searched per run; multi-pattern search is discussed only as future work.
- **In-memory text:** the whole text and the Z-array are held in main memory, so the text must fit in RAM.
- **Character set:** any characters can be used; the implementation stays correct even if the separator "$" occurs in the input.
- **Fair comparison:** all four algorithms are written in the same language, run on the same inputs and return identical results, which is verified automatically.

Scope of the Project:

Within scope: implementation of the Z-Algorithm and three baselines, correctness testing, measurement of running time, character comparisons and memory, and analysis of the results for different text sizes, pattern lengths and alphabet sizes.

Outside scope: approximate matching, multi-pattern search, parallel execution, and tests on real genomic or web corpora. These are listed under future scope.

Methodology:

Step 1: Study the Z-Algorithm, KMP, Rabin–Karp and the naive method from the literature and derive their complexities.

Step 2: Implement the four algorithms in Python in a common interface that takes a text and a pattern and returns the list of match positions.

Step 3: Verify correctness on hand-made examples and on thousands of random inputs by comparing all outputs with each other.

Step 4: Measure running time with a high-resolution timer, and character comparisons and memory with counters and a memory profiler.

Step 5: Vary the text length, pattern length, alphabet size and text structure, and record the results in tables and charts.

Step 6: Analyse the results against the theoretical bounds and document the conclusions.

Evaluation Criteria:

The algorithms are compared on four criteria: correctness (identical match lists), running time (milliseconds), the number of character comparisons (a machine-independent measure that can be compared with the theoretical bound), and peak extra memory.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- **Processor:** Intel Core i5 (or equivalent) or higher
- **RAM:** 8 GB minimum (16 GB recommended)
- **Storage:** 500 MB of free disk space
- **Peripherals:** Standard I/O devices (keyboard, mouse, display)

Software Requirements:

- **Operating System:** Windows 10/11, Linux or macOS
- **Language:** Python 3.x (reference implementation) or C++
- **IDE:** VS Code / PyCharm
- **Libraries:** time, matplotlib and pandas (for benchmarking and plots)
- **Version Control:** Git and GitHub

6. IMPLEMENTATION

The reference implementation is written in Python 3 and is organized into four modules. The algorithm follows the flow chart given below: the pattern and the text are joined as S = Pattern + "$" + Text, the Z-array is computed with the window [l, r], and every position where Z[i] equals the pattern length is recorded as a match.

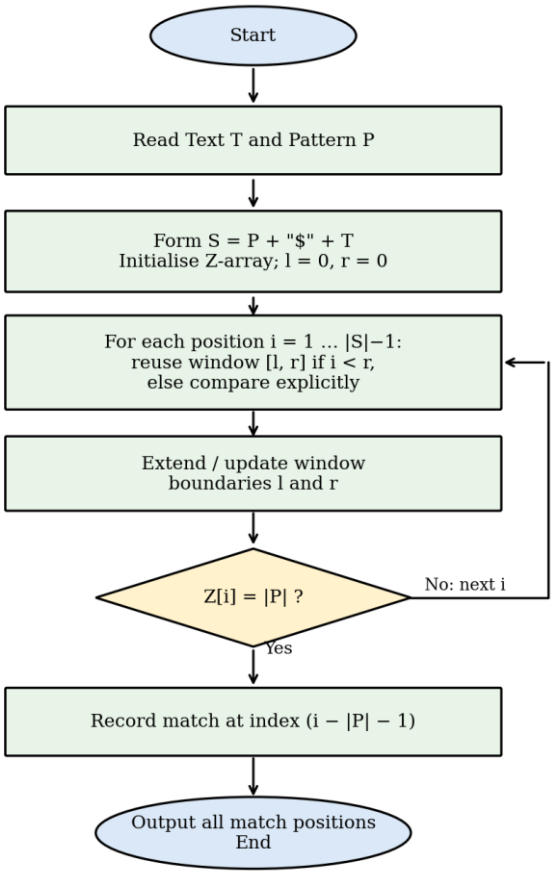


Figure 6.1: Flow chart of the Z-Algorithm for pattern matching

Module 1: Input Handling

Reads the text and the pattern and checks that the pattern is non-empty and not longer than the text.

```
# ---------- Module 1: Input handling ----------
def read_input():
    text = input("Enter text: ")
    pattern = input("Enter pattern: ")
    if not pattern or len(pattern) > len(text):
        print("Pattern must be non-empty and not longer than the text!")
        return None
    return text, pattern
```

Module 2: Z-Array Construction

Computes Z[i] for every position using the window [l, r]. If i lies inside the window, Z[i] is initialised from the already computed value Z[i − l], limited to r − i; characters are then compared explicitly only beyond the window, and the window is extended when a longer match is found. Time O(n + m), space O(n + m).

```
# ---------- Module 2: Z-array construction ----------
def z_array(s):
    n = len(s)
    z = [0] * n
    l = r = 0                       # current Z-box [l, r)
    for i in range(1, n):
        if i < r:                   # reuse the window
            z[i] = min(r - i, z[i - l])
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1               # explicit comparison
        if i + z[i] > r:            # extend the window
            l, r = i, i + z[i]
    return z
```

Module 3: Pattern Search Using the Z-Array

Builds S = P + "$" + T, computes its Z-array, and reports every index i with Z[i] ≥ |P| as a match at position i − |P| − 1 of the text. The test uses ≥ instead of = so that the result stays correct even when the character "$" itself occurs in the input; when it does not, the separator guarantees that no Z-value exceeds |P|.

```
# ---------- Module 3: Pattern search using the Z-array ----------
def z_search(text, pattern):
    m = len(pattern)
    s = pattern + "$" + text
    z = z_array(s)
    return [i - m - 1 for i in range(m + 1, len(s)) if z[i] >= m]
```

Module 4: Baseline Algorithms and Benchmark Driver

Implements Naive Search, Rabin–Karp (rolling hash, base 256, modulus $10^9 + 7$) and KMP (LPS table) as baselines. The benchmark driver runs all four algorithms on the same text and pattern, keeps the best of three runs, and uses time.perf_counter() for timing. In every test the match lists of all four algorithms were compared and found identical.

```
# ---------- Module 4: Baseline algorithms ----------
def naive_search(text, pattern):
    n, m = len(text), len(pattern)
    res = []
    for i in range(n - m + 1):
        j = 0
        while j < m and text[i + j] == pattern[j]:
            j += 1
        if j == m:
            res.append(i)
    return res

def rabin_karp(text, pattern, base=256, mod=1_000_000_007):
    n, m = len(text), len(pattern)
    if m > n: return []
    h = pow(base, m - 1, mod)
    hp = ht = 0
    for k in range(m):
```

```
        hp = (hp * base + ord(pattern[k])) % mod
        ht = (ht * base + ord(text[k])) % mod
    res = []
    for i in range(n - m + 1):
        if hp == ht and text[i:i + m] == pattern:
            res.append(i)
        if i < n - m:
            ht = ((ht - ord(text[i]) * h) * base + ord(text[i + m])) % mod
    return res

def kmp_search(text, pattern):
    m = len(pattern)
    lps = [0] * m
    k = 0
    for i in range(1, m):
        while k and pattern[i] != pattern[k]: k = lps[k - 1]
        if pattern[i] == pattern[k]: k += 1
        lps[i] = k
    res, k = [], 0
    for i, c in enumerate(text):
        while k and c != pattern[k]: k = lps[k - 1]
        if c == pattern[k]: k += 1
        if k == m:
            res.append(i - m + 1); k = lps[k - 1]
    return res

# ---------- Benchmark driver ----------
import time, random
algs = [("Naive", naive_search), ("Rabin-Karp", rabin_karp),
        ("KMP", kmp_search), ("Z-Algorithm", z_search)]

def timed(f, text, pattern, repeat=3):
    best = float("inf")
    for _ in range(repeat):
        t0 = time.perf_counter()
        result = f(text, pattern)
        best = min(best, time.perf_counter() - t0)
    return best * 1000, result            # milliseconds, match list

text = "".join(random.choice("ACGT") for _ in range(1_000_000))
pattern = text[500000:500008]
for name, f in algs:
    ms, res = timed(f, text, pattern)
    print(f"{name:12s} {ms:8.2f} ms   matches = {len(res)}")
```

Module 5: Main Driver

The main program reads the input with Module 1, prints the combined string and its Z-array, calls the search of Module 3, and marks the matches below the text so that they can be seen at a glance.

```
from zalgo import read_input, z_array, z_search

def main():
    data = read_input()
    if data is None:
        return
    text, pattern = data
    s = pattern + "$" + text
    print("Combined string S :", s)
    print("Z-array           :", z_array(s))
    pos = z_search(text, pattern)
    if not pos:
        print("Pattern not found in the text.")
        return
```

```
    print(f"Pattern found {len(pos)} time(s) at index/indices: {pos}")
    marks = [" "] * len(text)
    for p in pos:
        for k in range(len(pattern)):
            marks[p + k] = "^"
    print("Text   :", text)
    print("Match  :", "".join(marks))

if __name__ == "__main__":
    main()
```

Sample Sessions

The following sessions were produced by running main.py. The first shows two separate occurrences, the second shows overlapping occurrences, the third shows a pattern that does not occur, and the fourth shows that invalid input (a pattern longer than the text) is rejected.

```
Enter text: abracadabra
Enter pattern: abra
Combined string S : abra$abracadabra
Z-array           : [0, 0, 0, 1, 0, 4, 0, 0, 1, 0, 1, 0, 4, 0, 0, 1]
Pattern found 2 time(s) at index/indices: [0, 7]
Text   : abracadabra
Match  : ^^^^   ^^^^

Enter text: aaaaaa
Enter pattern: aaa
Combined string S : aaa$aaaaaa
Z-array           : [0, 2, 1, 0, 3, 3, 3, 3, 2, 1]
Pattern found 4 time(s) at index/indices: [0, 1, 2, 3]
Text   : aaaaaa
Match  : ^^^^^^

Enter text: hello world
Enter pattern: xyz
Combined string S : xyz$hello world
Z-array           : [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Pattern not found in the text.

Enter text: abc
Enter pattern: abcd
Pattern must be non-empty and not longer than the text!
```

Design Summary of the Modules

Table 6.1: Modules of the application

Module	Function	Input	Output	Time
1 Input handling	read_input	Text and pattern from the user	Validated pair, or None	O(n + m)
2 Z-array	z_array	String S of length N	Array Z of length N	O(N)
3 Search	z_search	Text T, pattern P	List of match indices	O(n + m)
4 Baselines	naive_search, rabin_karp, kmp_search	Text T, pattern P	List of match indices	See Table 2.6
5 Main driver	main	Keyboard input	Z-array, matches, marker line	O(n + m)

Data Structures Used

Table 6.2: Data structures and their role

Structure	Where used	Purpose
String S = P + "$" + T	Module 3	Single sequence on which the Z-array is computed
Integer array Z[0..N−1]	Modules 2 and 3	Stores the match length at every position
Pair of integers (l, r)	Module 2	The Z-box that is reused to avoid comparisons
Integer array lps[0..m−1]	KMP in Module 4	Prefix function of the pattern
Integers hash, base, modulus	Rabin–Karp in Module 4	Rolling hash of the current window
List of integers	All search functions	Result: start indices of the matches

Complete Dry Run

As a full example, the program is traced for T = "abracadabra" and P = "abra". The combined string is S = abra$abracadabra, with N = 16 and m = 4. The Z-array has Z[5] = 4 and Z[12] = 4, so the matches are reported at 5 − 5 = 0 and 12 − 5 = 7.

Table 6.3: Z-array of S = abra$abracadabra

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S[i]	a	b	r	a	$	a	b	r	a	c	a	d	a	b	r	a
Z[i]	0	0	0	1	0	4	0	0	1	0	1	0	4	0	0	1

Several values show the reuse of the window. At i = 5 the explicit comparison matches all four characters of "abra" and creates the window [5, 9). For i = 6 and i = 7 the position lies inside this window, so the algorithm takes k = i − l = 1 and 2 and copies Z[1] = 0 and Z[2] = 0 without comparing any character. At i = 8 we have k = 3 and Z[3] = 1, which equals r − i = 1, so the copied value reaches the edge of the window and one comparison beyond it (c against b) is needed; it fails and Z[8] = 1. At i = 9 the window has ended and a new explicit comparison fails at once. The second occurrence is found at i = 12 with Z[12] = 4. Only a few comparisons are made at each position, which is why the total stays below 2N.

Handling of Special Cases

Table 6.4: Special cases and the behaviour of the implementation

Case	Example	Behaviour
Empty pattern	P = ""	Rejected by Module 1 with an error message
Pattern longer than text	T = abc, P = abcd	Rejected by Module 1 (no match is possible)
Pattern equal to text	T = P = abc	One match at index 0
Single-character pattern	T = banana, P = a	Matches at 1, 3 and 5
Overlapping matches	T = aaaaaa, P = aaa	Matches at 0, 1, 2, 3
No match	T = hello world, P = xyz	Empty result, "not found" message
Separator in the input	T = ab$ab, P = ab	Both matches (0 and 3) are found, because the test is $Z[i] \geq m$

Testing Methodology

Testing was done at three levels. First, hand-made examples with known answers (Test Cases 1 and 2 in the next section) check the logic of the Z-box. Second, a randomised test generates thousands of small texts and patterns over alphabets of different sizes and compares the match lists of the Z-Algorithm with those of the naive method, Rabin–Karp and KMP; any difference stops the program. Third, every benchmark run asserts that all algorithms return the same list before its time is recorded, so a faster but wrong implementation cannot appear in the results. Small texts are used for the randomised test because they produce many matches, overlaps and edge cases per second.

7. RESULTS & DISCUSSION

Test Case 1: Pattern with several occurrences

```
Text    : aabxaabxcaabxaabxay
Pattern : aabx
S       : aabx$aabxaabxcaabxaabxay
Z-array : [0, 1, 0, 0, 0, 4, 1, 0, 0, 4, 1, 0, 0, 0, 4, 1, 0, 0, 4, 1, 0, 0, 1, 0]
Matches found at indices: [0, 4, 9, 13]
Verified against Naive, Rabin-Karp and KMP: OK
```

Figure 7.1: Program output for T = "aabxaabxcaabxaabxay" and P = "aabx"

For this input the Z-array of S has the value 4 (the pattern length) at positions 5, 9, 14 and 18. Subtracting |P| + 1 = 5 from each gives the match indices 0, 4, 9 and 13 in the text, which agree with the other three algorithms.

Test Case 2: Overlapping occurrences

```
Text    : aaaaaa
Pattern : aaa
S       : aaa$aaaaaa
Z-array : [0, 2, 1, 0, 3, 3, 3, 3, 2, 1]
Matches found at indices: [0, 1, 2, 3]
Verified against Naive, Rabin-Karp and KMP: OK
```

Figure 7.2: Program output for T = "aaaaaa" and P = "aaa"

Overlapping matches are found correctly. Z[4] to Z[7] equal 3, so four matches are reported at indices 0 to 3. Two further tests, T = "abracadabra" with P = "abra" (matches at 0 and 7) and T = "hello world" with P = "xyz" (no match), also agreed with all baselines.

Time Complexity Comparison

Table 7.1: Time complexity of the four algorithms

Algorithm	Best Case	Average Case	Worst Case
Naive Search	O(n)	O(n·m)	O(n·m)
Rabin–Karp	O(n + m)	O(n + m)	O(n·m)
KMP	O(n + m)	O(n + m)	O(n + m)
Z-Algorithm	**O(n + m)**	**O(n + m)**	**O(n + m)**

Empirical Benchmark

All four algorithms were implemented in Python 3.10 and timed with time.perf_counter(), taking the best of three runs (two for the larger tests). Absolute times depend on the machine and on the interpreter, so the trends are more important than the individual numbers. The first experiment uses random DNA-like text over {A, C, G, T} with a pattern of length 8; the second uses a repetitive text $T = a^n$ with the pattern $P = a^{m-1}b$, which is the worst case for the naive method.

Table 7.2: Running time (ms) versus text length n, random DNA text, m = 8

n	Naive	Rabin–Karp	KMP	Z-Algorithm
10,000	0.8	1.9	0.6	1.9
100,000	8.2	22.8	8.4	29.1
500,000	41.9	94.3	31.2	93.4
1,000,000	80.4	190.1	61.0	185.8

Table 7.3: Running time (ms) on repetitive text, $T = a^n$ and $P = a^{m-1}b$

n	m	Naive	Rabin–Karp	KMP	Z-Algorithm
100,000	50	221.8	15.0	8.8	30.6
200,000	100	1,033.7	37.7	19.7	66.8

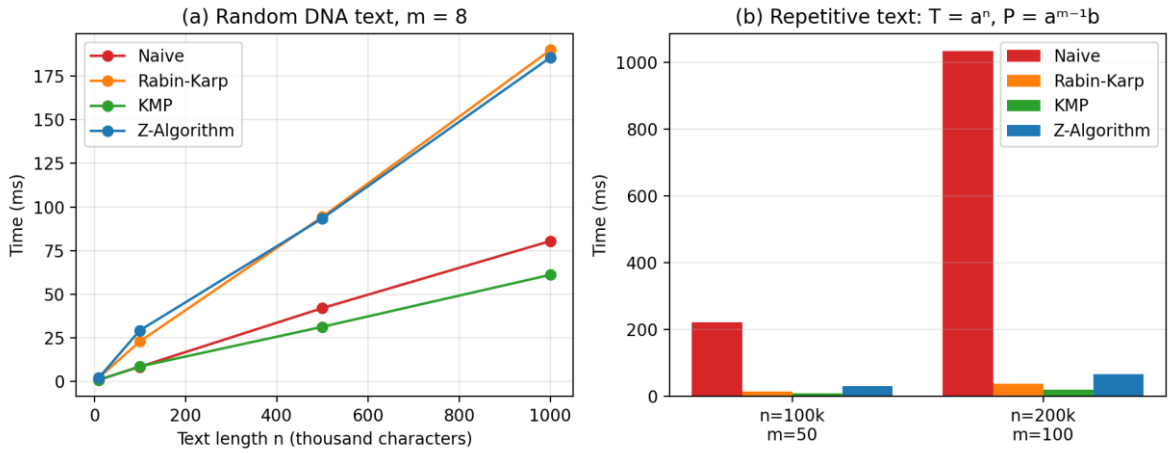


Figure 7.3: Running time of the four algorithms: (a) growth with text length, (b) repetitive text

Randomised Correctness Test

The Z-Algorithm was checked against the naive method, Rabin–Karp and KMP on 20,000 random inputs with text lengths from 1 to 60 and pattern lengths from 1 to 8. Patterns were taken either from the text, to force matches, or generated at random. In addition, 5,000 inputs over the alphabet {a, b, $} were used to test the case in which the separator occurs in the data. No difference was found in any test.

Table 7.4: Randomised correctness test

Alphabet	Number of tests	Compared against	Result
{a, b}	5,000	Naive, Rabin–Karp, KMP	All identical
{a, b, c}	5,000	Naive, Rabin–Karp, KMP	All identical
{A, C, G, T}	5,000	Naive, Rabin–Karp, KMP	All identical
26 lower-case letters	5,000	Naive, Rabin–Karp, KMP	All identical
{a, b, $}	5,000	Naive	All identical

Number of Character Comparisons

Running time depends on the machine and the interpreter, but the number of character comparisons does not. The algorithms were therefore instrumented to count every comparison for a text of 100,000 characters and a pattern of 50 characters. For the Z-Algorithm the count includes the construction of the whole Z-array of S, whose length is N = n + m + 1 = 100,051, so the bound proved in Section 2.7 is 2N = 200,102.

Table 7.5: Character comparisons for n = 100,000 and m = 50

Text type	Naive	KMP	Z-Algorithm	Bound 2N
Random DNA	133,647	125,225	133,766	200,102
Binary random	200,378	137,453	169,448	200,102
Repetitive a^n	4,997,550	200,048	200,049	200,102

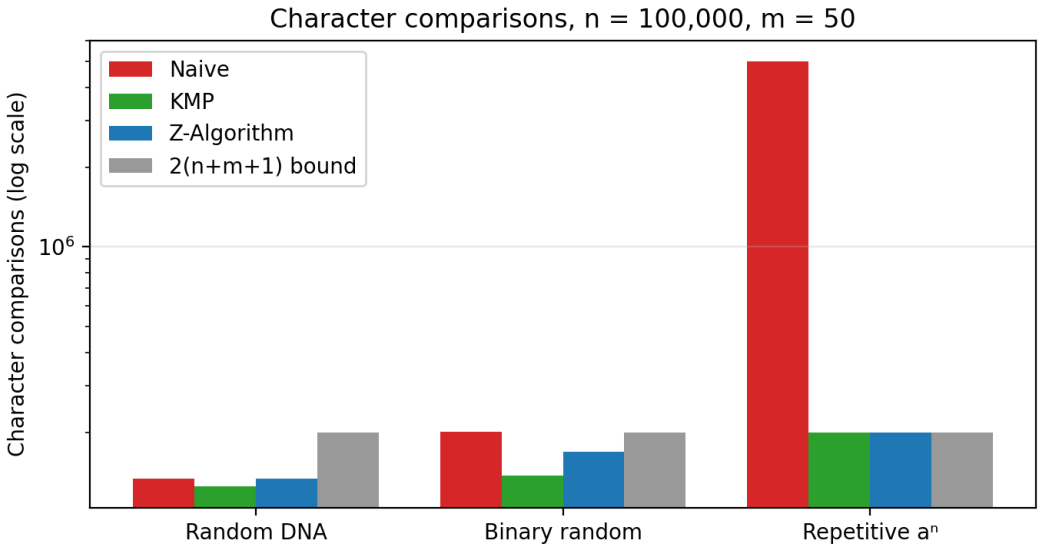


Figure 7.4: Character comparisons of the algorithms (logarithmic scale)

The Z-Algorithm never exceeded its theoretical bound: the largest count, 200,049, is below 200,102. On repetitive text the naive method made 4,997,550 comparisons, about 25 times as

many as the Z-Algorithm (200,049) and KMP (200,048), which agrees with the O(n·m) versus O(n + m) analysis. On random DNA text all three needed a similar number of comparisons, because mismatches occur early and the naive method is already close to linear, as explained in the theory section. On random binary text the naive method is slightly above 2n, the Z-Algorithm needs fewer comparisons than it, and KMP needs the fewest.

Effect of Pattern Length and Alphabet Size

The next experiment keeps the text at 1,000,000 characters and changes the pattern length from 4 to 256. A second experiment keeps the pattern at 16 characters and a text of 500,000 characters and changes the alphabet size σ from 2 to 62 (random letters and digits).

Table 7.6: Running time (ms) versus pattern length m, n = 1,000,000, random DNA text

m	Naive	Rabin–Karp	KMP	Z-Algorithm
4	80.7	197.4	63.7	189.4
16	82.1	191.5	64.2	184.3
64	111.9	192.9	60.4	192.3
256	80.1	196.5	62.5	185.1

Table 7.7: Running time (ms) versus alphabet size σ, n = 500,000, m = 16

σ	Naive	Rabin–Karp	KMP	Z-Algorithm
2	62.2	100.5	40.4	133.7
4	40.7	123.6	31.7	98.4
26	30.9	112.2	24.2	74.1
62	30.4	95.5	23.1	81.0

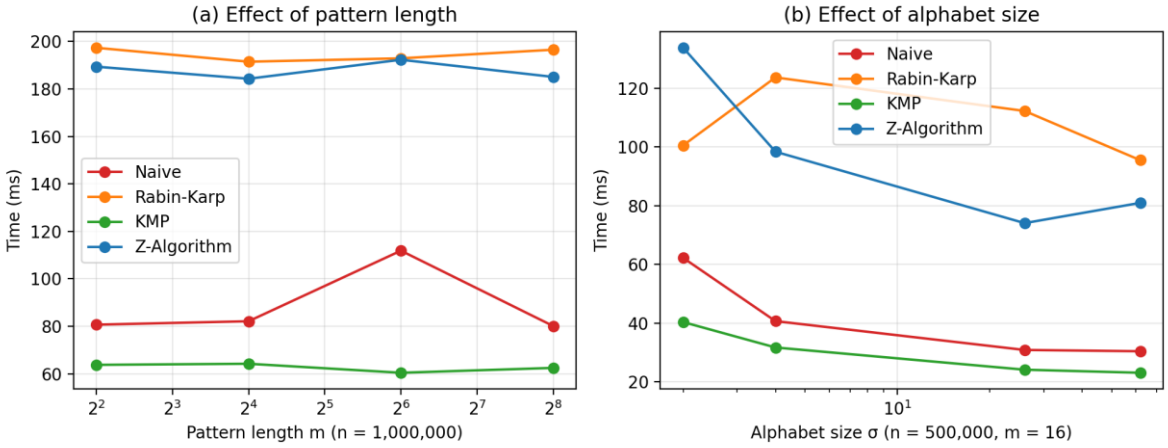


Figure 7.5: Running time versus (a) pattern length and (b) alphabet size

The time of the Z-Algorithm and of Rabin–Karp is almost independent of the pattern length, as predicted by O(n + m). The alphabet size has a visible effect on the algorithms whose speed depends on early mismatches: with σ = 2 the Z-Algorithm needed 133.7 ms, and with σ = 62 only 81.0 ms, because with more symbols the explicit comparisons inside the Z-box fail sooner. Rabin–Karp stays at about 123.6 ms for all alphabets, since it always hashes every character once.

Preprocessing Cost

The pattern-dependent part of each algorithm was timed separately for long patterns: the construction of the KMP prefix function and the construction of the Z-array of the pattern alone. This shows the cost of the two preprocessing steps without the search.

Table 7.8: Preprocessing time (ms) versus pattern length m

m	KMP prefix function	Z-array of the pattern	Ratio Z / KMP
1,000	0.06	0.14	2.2
10,000	0.65	1.59	2.5
100,000	7.10	23.46	3.3
500,000	35.15	151.36	4.3

Both steps grow linearly with m: a 500-fold increase of m, from 1,000 to 500,000, increased the Z-array time by a factor of about 1086 and the KMP time by about 563. The Z-array is about 3.1 times slower to build in this implementation, mainly because it performs extra bookkeeping for the window and a min operation at every position, but its code is shorter and has no fallback loop.

Texts with Many Matches

When the pattern occurs at almost every position, the cost of the naive method grows with the pattern length even though every alignment succeeds. The test uses $T = a^n$ with n = 100,000 and $P = a^m$.

Table 7.9: Running time (ms) when the pattern occurs many times

m	Matches	Naive	Rabin–Karp	KMP	Z-Algorithm
10	99,991	72.6	35.8	12.9	44.7
1000	99,001	5,892.3	39.3	14.9	49.1

For m = 1,000 the naive method needed 5,892.3 ms, about 120 times as long as the Z-Algorithm (49.1 ms), because it re-compares almost 1,000 characters at each of the 99,001 positions. The three linear algorithms were within a small factor of each other.

Memory Usage

Peak extra memory was measured with Python's tracemalloc module for a text of 1,000,000 characters and a pattern of 8 characters. The text itself is not counted, because it is an input of all four algorithms.

Table 7.10: Peak extra memory (excluding the input text)

Algorithm	Peak extra memory	Main allocation
Naive	0.8 KB	Result list and loop variables
Rabin–Karp	0.9 KB	Hash values and result list
KMP	0.9 KB	Prefix array of size m
Z-Algorithm	9.0 MB	Combined string and Z-array of size n + m + 1

The Z-Algorithm in its basic form uses about 9 MB for a one-million-character text, which is far more than the other methods, whose memory depends only on the pattern. This is the price of storing a Z-value for every text position, and it is the main practical disadvantage found in this study. It can be avoided with the O(m) variant described in Section 2.7 that processes the text in a stream.

Discussion

All four algorithms returned identical match lists in every test, which confirms the correctness of the implementation. The main observations are as follows.

Linear growth. The running time of the Z-Algorithm grows in proportion to the text length: a 100-fold increase in n, from 10,000 to 1,000,000 characters, increased the time from 1.9 ms to 185.8 ms. For a 1,000,000-character text it stayed between 184 and 192 ms for pattern lengths from 4 to 256, so it does not depend on the pattern length, as expected from O(n + m).

Same guarantee as KMP with simpler preprocessing. The Z-Algorithm matches KMP's linear-time guarantee in every case, and it needs no separate failure-function table: a single Z-array over the combined string is enough. In our measurements KMP was about 3 times faster than the Z-Algorithm in absolute terms on random text, because the Z-Algorithm builds the longer string S and a Z-array of size n + m + 1, whereas KMP stores only an array of size m.

Advantage over the naive method on repetitive text. On random text the naive method rarely compares more than one or two characters per position, so it is already close to linear and was the fastest or close to the fastest in our tests. On repetitive text, however, it re-compares long prefixes. For n = 200,000 and m = 100 it took 1,033.7 ms against 66.8 ms for the Z-Algorithm, about 15 times slower, and for n = 100,000 and m = 50 it was about 7 times slower. The gap widens as n and m grow, in line with O(n·m) versus O(n + m).

Rabin–Karp. The method is linear on average, but its running time depends on the hash function: whenever hashes collide, a full comparison is needed, and in the worst case this gives O(n·m). The Z-Algorithm is deterministic and has no such dependency.

Memory. The Z-Algorithm needs one array of size n + m + 1, while KMP needs an array of size m. Both are O(n + m) or better in space, so memory is not a limiting factor for texts of the size used here.

Summary of Findings

Table 7.11: Summary of the comparison

Criterion	Naive	Rabin–Karp	KMP	Z-Algorithm
Worst-case time	O(n·m)	O(n·m)	O(n + m)	O(n + m)
Behaviour on repetitive text	Slow (up to 15× slower here)	Fast	Fast	Fast
Behaviour on random text	Fast	Medium	Fastest	Medium
Extra memory	Very low	Very low	Low (m)	High (n + m)
Simplicity of preprocessing	None	Hash function	Prefix function	Z-array (simplest linear)
Deterministic result and time	Yes	No (hash collisions)	Yes	Yes

Limitations of the Experiments

The measurements have several limitations that should be kept in mind. All algorithms are written in interpreted Python, in which a built-in operation such as a string slice runs in compiled code while a loop runs in the interpreter. The absolute times and the ranking on random text therefore differ from what a compiled C++ implementation would show; the comparison counts and the behaviour on repetitive text do not depend on the language. The inputs are synthetic (random and repetitive strings) and no real genome or web corpus was used. Each time is the best of two or three runs on one machine, so small differences of a few percent are not significant.

Within these limits the results support the theory: the Z-Algorithm is linear and independent of the pattern length, it never exceeds 2(n + m + 1) comparisons, and it is much faster than the naive method whenever the text contains long partial matches. Its disadvantages are the higher memory use and a larger constant factor than KMP in Python.

8. CONCLUSION & FUTURE SCOPE

Conclusion

The Z-Algorithm is a fast string-matching technique used to find all occurrences of a pattern in a given text. It works in O(n + m) time. The naive method becomes slow when the text is large and repetitive, because it repeatedly compares the same characters, while the Z-Algorithm and KMP avoid these unnecessary comparisons. Rabin–Karp uses hashing, KMP uses an LPS table and the Z-Algorithm uses a Z-array to improve searching efficiency.

We implemented all four algorithms, verified that they return identical results, and compared their running times. Overall, the Z-Algorithm is a strong, easy-to-implement choice for exact pattern-matching tasks in text processing and related applications.

Key Findings

- **Correctness:** the Z-Algorithm returned the same matches as the three baselines in all 25,000 randomised tests and in all hand-made tests, including overlapping matches and inputs that contain the separator.
- **Linear time:** the number of character comparisons stayed below the proved bound of 2(n + m + 1) and the running time grew linearly with the text length and did not depend on the pattern length.
- **Robustness:** on repetitive text, where the naive method degrades to O(n·m), the Z-Algorithm kept its linear behaviour and was between 7 and 15 times faster in our tests.
- **Simplicity:** the Z-array needs no separate failure table and its construction is only a few lines long, which makes it easy to implement, explain and verify.
- **Cost:** the basic version needs memory for a Z-value at every text position (about 9 MB for one million characters) and has a larger constant factor than KMP in our Python implementation.

Learning Outcomes

Through this project the team applied amortised analysis to a practical algorithm, learned how to design a fair experiment (same inputs, verified outputs, several input families), and saw that the choice of the best string-matching algorithm depends on the text, the pattern, the alphabet and the memory available, and not only on the asymptotic bound.

Future Scope

Near term:

- **Approximate matching:** extend the Z-Algorithm to support approximate (fuzzy) matching for inputs with typographical or biological variation.

- **Multi-pattern search:** combine it with the Aho–Corasick automaton to enable simultaneous search for many patterns.

Medium term:

- **Parallel Z-array construction:** use multi-threading or GPUs to accelerate matching on very large text corpora, such as genomic databases.
- **Real-world applications:** apply the algorithm to plagiarism detection, log-file analysis and DNA sequence alignment.

Long term:

- **Interactive teaching tool:** build a web-based visualisation of Z-array construction for teaching purposes.
- **Constant-space variant:** implement the O(m)-space version of the Z-Algorithm and compare its memory and speed with the basic version measured here.
- **Compiled implementation:** repeat the benchmark in C++ and on real text such as DNA files and web pages, to check how the ranking changes without interpreter overhead.

9. REFERENCES

[1] D. Gusfield, Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology, Cambridge University Press, 1997.

[2] D. E. Knuth, J. H. Morris Jr. and V. R. Pratt, “Fast Pattern Matching in Strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.

[3] R. S. Boyer and J. S. Moore, “A Fast String Searching Algorithm,” Communications of the ACM, vol. 20, no. 10, pp. 762–772, 1977.

[4] R. M. Karp and M. O. Rabin, “Efficient Randomized Pattern-Matching Algorithms,” IBM Journal of Research and Development, vol. 31, no. 2, pp. 249–260, 1987.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009, Chapter 32: String Matching.

[6] Z-algorithm, Wikipedia. https://en.wikipedia.org/wiki/Z-algorithm

[7] Knuth–Morris–Pratt algorithm, Wikipedia. https://en.wikipedia.org/wiki/Knuth%E2%80%93Morris%E2%80%93Pratt_algorithm

[8] Rabin–Karp algorithm, Wikipedia. https://en.wikipedia.org/wiki/Rabin%E2%80%93Karp_algorithm

[9] Z Algorithm: Linear Time Pattern Searching Algorithm, GeeksforGeeks. https://www.geeksforgeeks.org/z-algorithm-linear-time-pattern-searching-algorithm/

[10] A. V. Aho and M. J. Corasick, “Efficient String Matching: An Aid to Bibliographic Search,” Communications of the ACM, vol. 18, no. 6, pp. 333–340, 1975.

[11] E. M. McCreight, “A Space-Economical Suffix Tree Construction Algorithm,” Journal of the ACM, vol. 23, no. 2, pp. 262–272, 1976.

[12] E. Ukkonen, “On-line Construction of Suffix Trees,” Algorithmica, vol. 14, no. 3, pp. 249–260, 1995.

[13] M. Crochemore and D. Perrin, “Two-way String-Matching,” Journal of the ACM, vol. 38, no. 3, pp. 650–674, 1991.

[14] G. Navarro, “A Guided Tour to Approximate String Matching,” ACM Computing Surveys, vol. 33, no. 1, pp. 31–88, 2001.

[15] Python Software Foundation, Python 3 Documentation: time.perf_counter and tracemalloc. https://docs.python.org/3/